Banco de Dados

Normalização — 2ª Forma Normal (2FN)

Recapitulando

Na aula anterior vimos:

- o que é **normalização**
- a **1ª Forma Normal (1FN)**
- valores **atômicos**
- eliminação de **grupos repetitivos**

Hoje: o que acontece quando a chave é **composta**.

Um problema para começar

Considere uma relação de notas:

matrícula	cod_disciplina	nome_aluno	nota
101	D01	Ana	8.5
101	D02	Ana	7.0
102	D01	João	9.0

Está na 1FN? **Sim** — todo valor é atômico.

Mas ainda há um problema.

Qual é a chave aqui?

Nenhum atributo sozinho identifica a linha.

A chave é composta:

```
(matrícula, cod_disciplina)
```

Só a combinação dos dois identifica uma nota específica.

Olhando com mais atenção

matrícula	cod_disciplina	nome_aluno	nota
101	D01	Ana	8.5
101	D02	Ana	7.0
102	D01	João	9.0

`nome_aluno` se repete para o mesmo `matrícula`, independente de `cod_disciplina`.

Será que `nome_aluno` realmente depende da chave inteira?

Dependência parcial

```
matrícula → nome_aluno
```

`nome_aluno` depende **só de parte** da chave (`matrícula`), não da chave composta inteira.

Isso é uma **dependência parcial** — e é exatamente o que a 2FN proíbe.

2ª Forma Normal

Uma relação está na **2ª Forma Normal** quando:

1. Está na **1FN**
2. **Todo atributo não-chave depende da chave inteira**, e não de apenas parte dela

> Se a chave não é composta, toda relação na 1FN já está automaticamente na 2FN.

Por que isso é um problema?

Com a estrutura atual:

- **Redundância:** o nome de Ana aparece repetido a cada disciplina
- **Anomalia de alteração:** mudar o nome de Ana exige atualizar várias linhas
- **Anomalia de inserção:** não dá para cadastrar um aluno sem matriculá-lo em uma disciplina
- **Anomalia de exclusão:** apagar a única nota de um aluno apaga também seu nome do sistema

Como resolver

Separar o que depende da **chave inteira** do que depende de **parte dela**.

```
ALUNO(matricula PK, nome_aluno)
MATRICULA(matricula FK, cod_disciplina FK, nota)
```

`nome_aluno` vai para a tabela onde `matrícula` é chave completa.
`nota` fica onde realmente pertence: depende dos dois campos.

Resultado

ALUNO

matrícula	nome_aluno
101	Ana
102	João

MATRICULA

matrícula	cod_disciplina	nota
101	D01	8.5
101	D02	7.0
102	D01	9.0

“ Agora o nome de Ana aparece **uma única vez**. ”

Outro exemplo: pedidos

pedido	produto	nome_cliente	quantidade
1001	Mouse	Ana	2
1001	Teclado	Ana	1
1002	Monitor	João	1

Chave: `(pedido, produto)`.

`nome_cliente` depende só de `pedido` → dependência parcial.

Colocando pedidos na 2FN

```
PEDIDO(pedido PK, nome_cliente)
ITEM_PEDIDO(pedido FK, produto FK, quantidade)
```

PEDIDO

pedido	nome_cliente
1001	Ana
1002	João

ITEM_PEDIDO

pedido	produto	quantidade
1001	Mouse	2
1001	Teclado	1
1002	Monitor	1

Mais um exemplo: estoque

cod_produto	cod_fornecedor	nome_produto	preco_fornecedor
P01	F1	Mouse	25.00
P01	F2	Mouse	27.50
P02	F1	Teclado	60.00

Chave: `(cod_produto, cod_fornecedor)`.

`nome_produto` depende só de `cod_produto` → dependência parcial.

Resolvendo estoque

```
PRODUTO(cod_produto PK, nome_produto)
FORNECIMENTO(cod_produto FK, cod_fornecedor FK, preco_fornecedor)
```

`preco_fornecedor` de fato depende dos dois: o mesmo mouse pode ter preços diferentes por fornecedor.

Atenção: 2FN só importa com chave composta

Se a chave é simples (um único atributo), não existe "parte da chave".

matrícula PK	nome	curso

Toda relação na 1FN com chave simples **já está na 2FN** automaticamente.

Como identificar dependência parcial

Pergunte, para cada atributo não-chave:

> "Esse atributo depende de **toda** a chave, ou eu consigo determinar o valor dele sabendo **só uma parte**?"

Se depender de só uma parte → dependência parcial → viola a 2FN.

Vamos praticar

Exercício 1

matrícula	cod_disciplina	nome_aluno	nota
101	D01	Ana	8.5
101	D02	Ana	7.0

Chave: `(matrícula, cod_disciplina)`. Essa relação está na 2FN?

A) Sim, já está na 1FN
B) Não, `nome_aluno` depende só de `matrícula`
C) Sim, `nota` depende da chave inteira
D) Não, falta uma chave estrangeira

Resposta 1

B) Não, porque `nome_aluno` depende apenas de `matrícula`, não da chave composta inteira.

`nota`, por outro lado, depende dos dois — está corretamente modelada.

Exercício 2

```
ITEM_VENDA(
    cod_venda PK,
    cod_produto PK,
    nome_produto,
    quantidade
)
```

O que viola a 2FN aqui?

A) `quantidade`, porque é numérico
B) `nome_produto`, porque depende só de `cod_produto`
C) `cod_venda`, porque é chave
D) Nada, a relação já está normalizada

Resposta 2

B) `nome_produto` **depende só de** `cod_produto`, não da chave composta `(cod_venda, cod_produto)`.

Correção:

```
PRODUTO(cod_produto PK, nome_produto)
ITEM_VENDA(cod_venda FK, cod_produto FK, quantidade)
```

Exercício 3

```
ALUNO(
    matricula PK,
    nome,
    curso
)
```

Essa relação viola a 2FN?

A) Sim, `curso` depende parcialmente
B) Sim, falta decompor `nome`
C) Não, a chave é simples
D) Não é possível saber sem mais dados

Resposta 3

C) Não — a chave é simples (`matrícula`), então não existe "parte da chave".

Toda relação na 1FN com chave simples já satisfaz a 2FN automaticamente.

Resumindo a 2FN

Para estar na 2FN, a relação precisa:

Estar na 1FN

Não ter dependência parcial
Todo atributo não-chave depende da **chave composta inteira**, nunca de apenas uma parte dela

Só é relevante quando a **chave é composta**.

E depois da 2FN?

Estar na 2FN elimina dependências parciais, mas ainda podem existir:

- dependências **transitivas** (um atributo não-chave depende de outro atributo não-chave)
- redundâncias que passam despercebidas

Por isso existe a próxima forma normal.

Próxima aula: 3FN

O que acontece quando um atributo não-chave depende de **outro atributo não-chave**, e não diretamente da chave?

```
matricula → cod_departamento → nome_departamento
```

Vamos descobrir por que isso também é um problema.

Fechando

2FN → elimina dependências parciais

Só se aplica a relações com chave composta

Todo atributo não-chave deve depender da chave inteira

Decomposição → separa o que depende de parte da chave

Próxima aula: **3FN**